# PYTHON PROGRAMMING

## Complete Case Study Solutions

Prepared using the assigned syllabus topics only



| | |
|---|---|
| **Name** | T. Thanuja Devi |
| **Roll Number** | 26000066 |
| **Section** | 1 |
| **Branch** | CSE |



Selected case studies: 7, 12, 13, 17, 23, 31, 38, 41, 42, and 49.

Each case study includes the problem statement, computational-thinking approach, algorithm, flowchart description, pseudocode, Python solution, and sample run.

# Contents


| Selection | Case No. | Case Study |
|---|---|---|
| 1 | 7 | Retail Billing with Multiple Discount Rules |
| 2 | 12 | Attendance Recovery Computation |
| 3 | 13 | ATM Transaction Sequence Simulation |
| 4 | 17 | Student Performance Trend Analysis |
| 5 | 23 | Student Timetable Conflict Detection |
| 6 | 31 | String-Based Examination Answer Evaluation |
| 7 | 38 | ATM Menu-Driven Banking Simulation |
| 8 | 41 | Nested List Classroom Occupancy Analysis |
| 9 | 42 | Student Marks Matrix Analysis |
| 10 | 49 | Tuple-Based Product Catalogue Audit |


## Syllabus boundary

The solutions use only introductory concepts from the assigned syllabus: computational thinking, algorithms, flowchart descriptions, pseudocode, variables, basic data types, operators, conditional statements, loops, strings, lists, nested lists, tuples, tuple operations, and unpacking. No external packages, classes, file handling, databases, or advanced programming constructs are required.

Flowcharts are provided as clear arrow-based flow sequences so they can be copied into standard flowchart symbols by hand.

# Case Study 7: Retail Billing with Multiple Discount Rules

**Selection:** 1 **Case study number:** 7

## 1. Problem statement

Prepare a bill for multiple products. Apply a 10% discount when the subtotal is at least ■5,000 and an additional 5% discount when the customer is a member. Calculate the final amount.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Variables, numeric data types, operators, conditional statements, loops, lists.

## 3. Algorithm

1. Start.

2. Read the number of products and whether the customer is a member.

3. Use a loop to read each product name, quantity, and unit price; store the line totals.

4. Add all line totals to obtain the subtotal.

5. If the subtotal is at least ■5,000, calculate a 10% discount; otherwise use zero.

6. If the customer is a member, calculate an additional 5% discount on the subtotal after the first discount.

7. Calculate the final amount and display the bill summary.

8. Stop.

## 4. Flowchart (text sequence)

Start → Read products and membership → Calculate subtotal → Check subtotal threshold → Apply applicable discounts → Display bill → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT n, member
  subtotal = 0
  FOR each product from 1 to n
      INPUT name, quantity, price
      line_total = quantity * price
      subtotal = subtotal + line_total
      STORE product details and line_total
  END FOR
  IF subtotal >= 5000 THEN
      discount1 = subtotal * 0.10
  ELSE
      discount1 = 0
  END IF
  amount_after_discount = subtotal - discount1
  IF member == "yes" THEN
      discount2 = amount_after_discount * 0.05
  ELSE
      discount2 = 0
  END IF
  final_amount = amount_after_discount - discount2
  DISPLAY bill details and final_amount
END
```

## 6. Python program

```
products = []
n = int(input("Number of products: "))
member = input("Member customer? (yes/no): ").strip().lower()
subtotal = 0.0

for i in range(n):
```

```
    print("Product", i + 1)
    name = input("Name: ")
    quantity = int(input("Quantity: "))
    price = float(input("Unit price: "))
    line_total = quantity * price
    products.append((name, quantity, price, line_total))
    subtotal += line_total

if subtotal >= 5000:
    discount1 = subtotal * 0.10
else:
    discount1 = 0.0

after_first = subtotal - discount1
if member == "yes":
    discount2 = after_first * 0.05
else:
    discount2 = 0.0

final_amount = after_first - discount2

print("\n--- BILL ---")
for name, quantity, price, line_total in products:
    print(name, "| Qty:", quantity, "| Unit:", round(price, 2),
          "| Total:", round(line_total, 2))
print("Subtotal:", round(subtotal, 2))
print("Threshold discount:", round(discount1, 2))
print("Member discount:", round(discount2, 2))
print("Final amount:", round(final_amount, 2))
```

## 7. Sample run / expected result

Number of products: 2\nProduct 1: Notebook, quantity 10, unit price 100\nProduct 2: Bag, quantity 1, unit price 4200\nMember customer? yes\nSubtotal: ■5200.00\nThreshold discount: ■520.00\nMember discount: ■234.00\nFinal amount: ■4446.00

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 12: Attendance Recovery Computation

**Selection:** 2 **Case study number:** 12

## 1. Problem statement

For each student, read classes held and classes attended. Calculate attendance percentage and determine how many additional classes the student must attend consecutively to reach a target percentage, assuming no new classes are missed.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Variables, arithmetic operators, conditional statements, loops, lists.

## 3. Algorithm

1. Start and read the target percentage.
2. Read the number of students.
3. For each student, read name, classes held, and classes attended.
4. Calculate attendance percentage; handle zero classes held.
5. If attendance is already at or above the target, required additional classes are zero.
6. Otherwise increase the hypothetical attended and held counts together until the target is reached.
7. Display each student's percentage and required additional classes.
8. Stop.

## 4. Flowchart (text sequence)

Start → Read target and student data → Validate counts → Calculate percentage → Loop until target reached → Display result → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT target
  INPUT n
  FOR each student
      INPUT name, held, attended
      IF held <= 0 THEN
          DISPLAY "Attendance unavailable"
      ELSE
          percentage = attended * 100 / held
          extra = 0
          WHILE (attended + extra) * 100 / (held + extra) < target
              extra = extra + 1
          END WHILE
          DISPLAY name, percentage, extra
      END IF
  END FOR
END
```

## 6. Python program

```
target = float(input("Target attendance percentage (e.g. 75): "))
n = int(input("Number of students: "))
records = []

for i in range(n):
    name = input("Student name: ")
    held = int(input("Classes held: "))
    attended = int(input("Classes attended: "))

    if held <= 0:
```

```
            records.append((name, None, None))
            continue
        if attended < 0 or attended > held:
            print("Invalid attendance counts; enter this student again.")
            continue

        percentage = attended * 100 / held
        extra = 0
        while (attended + extra) * 100 / (held + extra) < target:
            extra += 1
        records.append((name, percentage, extra))

print("\n--- ATTENDANCE RECOVERY ---")
for name, percentage, extra in records:
    if percentage is None:
        print(name, ": attendance unavailable (classes held must be positive)")
    else:
        print(name, ": current =", round(percentage, 2), "%",
              "| extra classes needed =", extra)
```

## 7. Sample run / expected result

Target attendance: 75\nName: Ravi; held: 20; attended: 12\nCurrent attendance = 60.0%\nExtra classes needed = 12

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 13: ATM Transaction Sequence Simulation

**Selection:** 3 **Case study number:** 13

## 1. Problem statement

Simulate a sequence of ATM deposits, withdrawals, and balance checks. Reject invalid amounts and withdrawals exceeding the available balance.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Variables, numeric types, operators, conditionals, loops, lists.

## 3. Algorithm

1. Start and read the opening balance.
2. Repeatedly display the transaction menu.
3. Read the selected operation until the user chooses Exit.
4. For deposit, accept only a positive amount and add it to the balance.
5. For withdrawal, accept only a positive amount and ensure sufficient funds.
6. For balance inquiry, display the current balance.
7. Record each successful transaction in a list and display it at the end.
8. Stop.

## 4. Flowchart (text sequence)

Start → Enter opening balance → Menu loop → Perform selected transaction with checks → Update and record balance → Exit and show history → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT balance
  transactions = empty list
  REPEAT
      DISPLAY 1.Deposit 2.Withdraw 3.Balance 4.Exit
      INPUT choice
      IF choice == 1 THEN
          INPUT amount
          IF amount > 0 THEN
              balance = balance + amount
              APPEND transaction
          ELSE DISPLAY "Invalid amount"
          END IF
      ELSE IF choice == 2 THEN
          INPUT amount
          IF amount <= 0 THEN DISPLAY "Invalid amount"
          ELSE IF amount > balance THEN DISPLAY "Insufficient funds"
          ELSE
              balance = balance - amount
              APPEND transaction
          END IF
      ELSE IF choice == 3 THEN DISPLAY balance
      ELSE IF choice != 4 THEN DISPLAY "Invalid choice"
      END IF
  UNTIL choice == 4
  DISPLAY transactions and balance
END
```

## 6. Python program

```
balance = float(input("Opening balance: "))
transactions = []
```

```
while True:
    print("\n1. Deposit  2. Withdraw  3. Check balance  4. Exit")
    choice = input("Choose an option: ").strip()

    if choice == "1":
        amount = float(input("Deposit amount: "))
        if amount > 0:
            balance += amount
            transactions.append(("Deposit", amount, balance))
            print("Deposit successful.")
        else:
            print("Amount must be positive.")
    elif choice == "2":
        amount = float(input("Withdrawal amount: "))
        if amount <= 0:
            print("Amount must be positive.")
        elif amount > balance:
            print("Insufficient funds.")
        else:
            balance -= amount
            transactions.append(("Withdrawal", amount, balance))
            print("Withdrawal successful.")
    elif choice == "3":
        print("Current balance:", round(balance, 2))
    elif choice == "4":
        break
    else:
        print("Invalid option. Try again.")

print("\n--- TRANSACTION HISTORY ---")
for kind, amount, remaining in transactions:
    print(kind, amount, "| Balance after transaction:", round(remaining, 2))
print("Closing balance:", round(balance, 2))
```

## 7. Sample run / expected result

Opening balance: 1000\nDeposit 500 → Balance 1500\nWithdraw 200 → Balance 1300\nClosing balance: ■1300.00

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 17: Student Performance Trend Analysis

**Selection:** 4 **Case study number:** 17

## 1. Problem statement

Store each student's marks across multiple tests, calculate the average, and identify whether their marks are improving, declining, or unchanged from the first test to the last.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Lists, loops, arithmetic operators, conditionals, nested lists.

## 3. Algorithm

1. Read the number of students and tests.
2. For each student, read the name and marks for every test.
3. Store the marks in a list and calculate the average.
4. Compare the last test mark with the first test mark.
5. Classify the trend as Improving, Declining, or Unchanged.
6. Display average and trend for each student.
7. Stop.

## 4. Flowchart (text sequence)

Start → Read student/test counts → Collect marks in lists → Calculate average → Compare first and last marks → Display trend → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT student_count, test_count
  FOR each student
      INPUT name
      marks = empty list
      FOR each test
          INPUT mark
          APPEND mark to marks
      END FOR
      average = SUM(marks) / test_count
      IF marks[last] > marks[first] THEN trend = "Improving"
      ELSE IF marks[last] < marks[first] THEN trend = "Declining"
      ELSE trend = "Unchanged"
      DISPLAY name, average, trend
  END FOR
END
```

## 6. Python program

```
student_count = int(input("Number of students: "))
test_count = int(input("Number of tests per student: "))
students = []

if student_count <= 0 or test_count <= 0:
    print("Students and tests must both be at least 1.")
else:
    for i in range(student_count):
        name = input("Student name: ")
        marks = []
        for j in range(test_count):
            mark = float(input("Mark for test " + str(j + 1) + ": "))
            marks.append(mark)

        average = sum(marks) / len(marks)
```

```
        if marks[-1] > marks[0]:
            trend = "Improving"
        elif marks[-1] < marks[0]:
            trend = "Declining"
        else:
            trend = "Unchanged"

        students.append([name, marks, average, trend])

    print("\n--- PERFORMANCE ANALYSIS ---")
    for name, marks, average, trend in students:
        print(name, "| Marks:", marks, "| Average:", round(average, 2),
              "| Trend:", trend)
```

## 7. Sample run / expected result

Name: Anu\nMarks: [65, 72, 81]\nAverage: 72.67\nTrend: Improving

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 23: Student Timetable Conflict Detection

**Selection:** 5 **Case study number:** 23

## 1. Problem statement

Read a list of class timetable entries with day, start time, end time, and subject. Detect overlapping classes scheduled on the same day.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Strings, lists, nested lists, loops, conditional statements, operators.

## 3. Algorithm

1. Read the number of timetable entries.
2. For each entry, read day, start time, end time, and subject.
3. Convert times entered as HH:MM into minutes after midnight.
4. Compare every pair of entries.
5. If two entries are on the same day and each starts before the other ends, report a conflict.
6. Otherwise report that no conflicts were found.
7. Stop.

## 4. Flowchart (text sequence)

Start → Read timetable entries → Convert times → Compare pairs on same day → Report overlaps or no conflicts → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT n
  timetable = empty list
  FOR i = 1 TO n
      INPUT day, start, end, subject
      CONVERT start and end to minutes
      APPEND entry to timetable
  END FOR
  conflict_found = FALSE
  FOR i = 0 TO n - 1
      FOR j = i + 1 TO n - 1
          IF timetable[i].day == timetable[j].day AND
             timetable[i].start < timetable[j].end AND
             timetable[j].start < timetable[i].end THEN
              DISPLAY both conflicting entries
              conflict_found = TRUE
          END IF
      END FOR
  END FOR
  IF conflict_found == FALSE THEN DISPLAY "No conflicts"
END
```

## 6. Python program

```
def to_minutes(time_text):
    parts = time_text.split(":")
    hour = int(parts[0])
    minute = int(parts[1])
    return hour * 60 + minute

n = int(input("Number of timetable entries: "))
timetable = []

for i in range(n):
```

```
        print("Entry", i + 1)
        day = input("Day: ").strip().lower()
        start_text = input("Start time (HH:MM): ")
        end_text = input("End time (HH:MM): ")
        subject = input("Subject: ")
        start = to_minutes(start_text)
        end = to_minutes(end_text)

        if start >= end:
            print("This entry has an invalid time range and is skipped.")
            continue
        timetable.append([day, start, end, subject, start_text, end_text])

found = False
for i in range(len(timetable)):
    for j in range(i + 1, len(timetable)):
        a = timetable[i]
        b = timetable[j]
        if a[0] == b[0] and a[1] < b[2] and b[1] < a[2]:
            print("Conflict:", a[3], a[0], a[4] + "-" + a[5],
                  "overlaps with", b[3], b[0], b[4] + "-" + b[5])
            found = True

if not found:
    print("No timetable conflicts found.")
```

## 7. Sample run / expected result

Entry 1: Monday, 09:00–10:00, Maths\nEntry 2: Monday, 09:30–10:30, Physics\nConflict: Maths overlaps with Physics

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 31: String-Based Examination Answer Evaluation

**Selection:** 6 **Case study number:** 31

## 1. Problem statement

Evaluate a student's answer against an expected answer using a simple string-matching approach. Normalize capitalization and extra spaces, then report exact match or mismatch. This is a basic demonstration, not a full semantic grading system.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Strings, loops, conditional statements, operators, lists.

## 3. Algorithm

1. Read the expected answer and student's answer.
2. Convert both strings to lowercase and remove leading/trailing spaces.
3. Split each string into words and join them with single spaces to normalize repeated spaces.
4. Compare the normalized strings.
5. Display Exact Match or Mismatch.
6. Optionally show common words to support simple feedback.
7. Stop.

## 4. Flowchart (text sequence)

Start → Read expected and student answers → Normalize strings → Compare → Display match/mismatch → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT expected_answer, student_answer
  expected = NORMALIZE(expected_answer)
  student = NORMALIZE(student_answer)
  IF expected == student THEN
      DISPLAY "Exact match"
  ELSE
      DISPLAY "Mismatch"
      expected_words = SPLIT(expected)
      student_words = SPLIT(student)
      DISPLAY common words
  END IF
END
```

## 6. Python program

```
expected_answer = input("Enter the expected answer: ")
student_answer = input("Enter the student's answer: ")

expected = " ".join(expected_answer.lower().split())
student = " ".join(student_answer.lower().split())

if expected == student:
    print("Result: Exact match")
else:
    print("Result: Mismatch")
    expected_words = expected.split()
    student_words = student.split()
    common_words = []
    for word in student_words:
        if word in expected_words and word not in common_words:
            common_words.append(word)
    print("Common words:", common_words)
    print("Note: This simple program checks text, not meaning.")
```

## 7. Sample run / expected result

Expected: Python uses lists\nStudent: python uses lists\nResult: Exact match

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 38: ATM Menu-Driven Banking Simulation

**Selection:** 7 **Case study number:** 38

## 1. Problem statement

Create a menu-driven banking program with balance inquiry, deposit, withdrawal, and transaction history. This is a classroom simulation and does not connect to a real bank.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Variables, operators, conditionals, loops, menu design, lists.

## 3. Algorithm

1. Initialize a sample balance and an empty transaction list.
2. Display the banking menu repeatedly.
3. Perform the selected action.
4. Validate deposits and withdrawals; do not allow negative or over-balance withdrawals.
5. Record successful deposits and withdrawals.
6. Display transaction history when requested.
7. Exit when the user selects Exit.

## 4. Flowchart (text sequence)

Start → Initialize balance → Display menu → Execute chosen operation → Update/record transaction → Repeat or exit → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  balance = 0
  transactions = empty list
  REPEAT
      DISPLAY menu
      INPUT choice
      IF choice == 1: DISPLAY balance
      ELSE IF choice == 2:
          INPUT amount
          IF amount > 0: balance = balance + amount; record
          ELSE DISPLAY invalid amount
      ELSE IF choice == 3:
          INPUT amount
          IF amount > 0 AND amount <= balance:
              balance = balance - amount; record
          ELSE DISPLAY error
      ELSE IF choice == 4: DISPLAY transactions
      ELSE IF choice != 5: DISPLAY invalid choice
  UNTIL choice == 5
END
```

## 6. Python program

```
balance = 0.0
transactions = []

while True:
    print("\n--- BANKING MENU ---")
    print("1. Balance inquiry")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Transaction history")
    print("5. Exit")
    choice = input("Select: ").strip()
```

```
if choice == "1":
    print("Available balance:", round(balance, 2))
elif choice == "2":
    amount = float(input("Deposit amount: "))
    if amount > 0:
        balance += amount
        transactions.append(("Deposit", amount))
        print("Deposit completed.")
    else:
        print("Enter a positive amount.")
elif choice == "3":
    amount = float(input("Withdrawal amount: "))
    if amount <= 0:
        print("Enter a positive amount.")
    elif amount > balance:
        print("Insufficient balance.")
    else:
        balance -= amount
        transactions.append(("Withdrawal", amount))
        print("Withdrawal completed.")
elif choice == "4":
    if len(transactions) == 0:
        print("No transactions yet.")
    else:
        for number, transaction in enumerate(transactions, start=1):
            kind, amount = transaction
            print(number, kind, round(amount, 2))
elif choice == "5":
    print("Thank you. Final balance:", round(balance, 2))
    break
else:
    print("Invalid choice.")
```

## 7. Sample run / expected result

Starting balance: ■0\nDeposit ■1000\nWithdraw ■250\nBalance inquiry: ■750\nFinal balance: ■750

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 41: Nested List Classroom Occupancy Analysis

**Selection:** 8 **Case study number:** 41

## 1. Problem statement

Represent classroom occupancy as a nested list. Each row represents a classroom and contains the number of students in different sessions. Calculate total occupancy per classroom and identify the highest occupancy.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Nested lists, loops, arithmetic operators, conditional statements.

## 3. Algorithm

1. Read the number of classrooms and sessions.

2. Use nested loops to collect student counts in a two-dimensional list.

3. Calculate the total occupancy of each classroom by summing its row.

4. Compare totals to find the classroom with the highest occupancy.

5. Display the matrix, row totals, and highest occupancy.

6. Stop.

## 4. Flowchart (text sequence)

Start → Read room/session counts → Build nested list → Sum each row → Find maximum total → Display analysis → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT rooms, sessions
  occupancy = empty nested list
  FOR each room
      row = empty list
      FOR each session
          INPUT count
          APPEND count to row
      END FOR
      APPEND row to occupancy
  END FOR
  highest = -1
  FOR each room row
      total = SUM(row)
      DISPLAY total
      IF total > highest THEN
          highest = total
          busiest_room = room
      END IF
  END FOR
  DISPLAY busiest_room, highest
END
```

## 6. Python program

```
rooms = int(input("Number of classrooms: "))
sessions = int(input("Number of sessions per classroom: "))
occupancy = []

if rooms <= 0 or sessions <= 0:
    print("Classrooms and sessions must both be at least 1.")
else:
    for i in range(rooms):
        row = []
        print("Classroom", i + 1)
        for j in range(sessions):
```

```
            count = int(input("Students in session " + str(j + 1) + ": "))
            row.append(count)
        occupancy.append(row)

totals = []
for row in occupancy:
    totals.append(sum(row))

print("Occupancy matrix:", occupancy)
for i in range(len(totals)):
    print("Classroom", i + 1, "total occupancy:", totals[i])

highest = max(totals)
busiest_rooms = []
for i in range(len(totals)):
    if totals[i] == highest:
        busiest_rooms.append(i + 1)
print("Highest total occupancy:", highest)
print("Classroom(s) with highest total:", busiest_rooms)
```

## 7. Sample run / expected result

Occupancy matrix: [[25, 28], [30, 27], [20, 22]]\nTotals: 53, 57, 42\nHighest total: 57 in Classroom 2

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 42: Student Marks Matrix Analysis

**Selection:** 9 **Case study number:** 42

## 1. Problem statement

Store marks for multiple students across multiple subjects in a matrix. Calculate each student's total and average, and calculate the average mark for each subject.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Nested lists, loops, arithmetic operators, conditional statements.

## 3. Algorithm

1. Read student and subject counts.
2. Read student names and subject names.
3. Use nested loops to collect marks into a matrix.
4. Calculate row totals and averages for each student.
5. Calculate each column's average for each subject.
6. Display the matrix and summary results.
7. Stop.

## 4. Flowchart (text sequence)

Start → Read names and dimensions → Build marks matrix → Calculate student totals/averages → Calculate subject averages → Display → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  INPUT students, subjects
  INPUT student_names and subject_names
  marks = empty nested list
  FOR each student
      row = empty list
      FOR each subject
          INPUT mark
          APPEND mark to row
      END FOR
      APPEND row to marks
  END FOR
  FOR each student row
      DISPLAY SUM(row), SUM(row) / subjects
  END FOR
  FOR each subject column
      total = 0
      FOR each student row
          total = total + marks[student][subject]
      END FOR
      DISPLAY total / students
  END FOR
END
```

## 6. Python program

```
student_count = int(input("Number of students: "))
subject_count = int(input("Number of subjects: "))

if student_count <= 0 or subject_count <= 0:
    print("Students and subjects must both be at least 1.")
else:
    student_names = []
    subject_names = []
```

```
for i in range(student_count):
    student_names.append(input("Student name: "))
for j in range(subject_count):
    subject_names.append(input("Subject name: "))

marks = []
for i in range(student_count):
    row = []
    print("Enter marks for", student_names[i])
    for j in range(subject_count):
        row.append(float(input(subject_names[j] + ": ")))
    marks.append(row)

print("\n--- MARKS MATRIX ---")
print("Subjects:", subject_names)
for i in range(student_count):
    total = sum(marks[i])
    average = total / subject_count
    print(student_names[i], marks[i], "| Total:", total,
          "| Average:", round(average, 2))

print("\n--- SUBJECT AVERAGES ---")
for j in range(subject_count):
    total = 0
    for i in range(student_count):
        total += marks[i][j]
    print(subject_names[j], "average:", round(total / student_count, 2))
```

## 7. Sample run / expected result

Subjects: Maths, Science\nAnu: [80, 90] Total: 170 Average: 85.0\nBalu: [70, 80] Total: 150 Average: 75.0\nSubject averages: Maths 75.0, Science 85.0

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Case Study 49: Tuple-Based Product Catalogue Audit

**Selection:** 10 **Case study number:** 49

## 1. Problem statement

Store product records as tuples containing product ID, name, price, and stock. Display the catalogue, find a product by ID, and report products that need restocking.

## 2. Computational thinking and syllabus concepts

**Decomposition:** Break the task into input, processing, decisions, and output. **Pattern recognition:** Identify repeated calculations or comparisons. **Abstraction:** Keep only the data needed for the result. **Concepts used:** Tuples, tuple operations, unpacking, loops, conditional statements.

## 3. Algorithm

1. Create or read product records as tuples.

2. Display each record using tuple unpacking.

3. Read a product ID and search for it in the catalogue.

4. If found, display its details; otherwise report not found.

5. Read the minimum stock threshold and list products below that threshold.

6. Stop.

## 4. Flowchart (text sequence)

Start → Read/display tuple catalogue → Search by ID → Report result → Check stock threshold → Display restock list → Stop

Suggested symbols: oval for Start/Stop, parallelogram for input/output, rectangle for processing, and diamond for a decision or loop condition.

## 5. Pseudocode

```
START
  catalogue = list of product tuples
  FOR each tuple in catalogue
      UNPACK id, name, price, stock
      DISPLAY product details
  END FOR
  INPUT search_id
  found = FALSE
  FOR each tuple in catalogue
      UNPACK id, name, price, stock
      IF id == search_id THEN DISPLAY details; found = TRUE
  END FOR
  IF found == FALSE THEN DISPLAY "Not found"
  INPUT threshold
  FOR each tuple in catalogue
      UNPACK id, name, price, stock
      IF stock < threshold THEN DISPLAY product for restocking
  END FOR
END
```

## 6. Python program

```
catalogue = [
    (101, "Notebook", 45.0, 12),
    (102, "Pen", 10.0, 4),
    (103, "School Bag", 650.0, 2),
    (104, "Pencil", 5.0, 20)
]

print("--- PRODUCT CATALOGUE ---")
for product in catalogue:
    product_id, name, price, stock = product  # tuple unpacking
    print("ID:", product_id, "| Name:", name,
          "| Price:", price, "| Stock:", stock)

search_id = int(input("Enter product ID to search: "))
```

```
found = False
for product_id, name, price, stock in catalogue:
    if product_id == search_id:
        print("Product found:", product_id, name, price, stock)
        found = True
        break

if not found:
    print("Product ID not found.")

threshold = int(input("Minimum desired stock: "))
print("\n--- RESTOCKING LIST ---")
restock_found = False
for product_id, name, price, stock in catalogue:
    if stock < threshold:
        print(product_id, name, "- current stock:", stock)
        restock_found = True

if not restock_found:
    print("No products need restocking.")
```

## 7. Sample run / expected result

Catalogue contains Notebook (stock 12), Pen (stock 4), School Bag (stock 2), Pencil (stock 20).\nSearch ID 103 → Product found: School Bag\nMinimum stock 5 → Pen and School Bag need restocking.

## 8. Testing note

Test normal inputs, boundary values, and invalid inputs. Check that the output follows the rules stated in the problem. The sample run is illustrative; enter the values in the order requested by the program.

# Final Review

Before submitting, run each program in a Python interpreter and attach handwritten or software-drawn flowcharts if your instructor requires standard symbols. Confirm the case-study titles and the discount/attendance rules with your course handout, since the supplied list contained titles but not the original detailed case specifications.

## Syllabus checklist

- Computational thinking, decomposition, abstraction, and pattern recognition
- Algorithms, flowchart sequences, and pseudocode
- Variables, integers, floating-point numbers, strings, and Boolean conditions
- Arithmetic, comparison, and logical operators
- if / elif / else conditional statements
- for and while loops
- String normalization and splitting
- Lists and nested lists
- Tuples, tuple operations, and unpacking
- Input validation and sample testing